EXPERIMENT - 5

Aim

To design a Hamming Code sender utility and to generate an error-detecting bitstream using even parity.

Procedure

1. Input the binary data
2. Find check bits using 2r≥m+r+12^r \ge m + r + 12r≥m+r+1
3. Place check bits at positions 1, 2, 4, 8...
4. Fill remaining positions with data bits
5. Group bits based on position
6. Set check bits for even parity
7. Combine to get final encoded data

Program

```
def calculate_parity_bits(m):

    p = 0

    while (2 ** p) < m + p + 1:

        p += 1

    return p



def position_parity_bits(data, p):

    j = 0

    k = 1

    m = len(data)

    result = ''
```

```
    for i in range(1, m + p + 1):
        if i == 2 ** j:
            result += '0'
            j += 1
        else:
            result += data[-k]
            k += 1
    return result[::-1]


def calculate_parity_values(code, p):
    n = len(code)
    for i in range(p):
        val = 0
        for j in range(1, n + 1):
            if j & (2 ** i) == (2 ** i):
                val = val ^ int(code[-j])
        code = code[:n-(2**i)] + str(val) + code[n-(2**i)+1:]
    return code


def detect_correct_error(code, p):
    n = len(code)
    error_pos = 0
```

```
    for i in range(p):
        val = 0
        for j in range(1, n + 1):
            if j & (2 ** i) == (2 ** i):
                val = val ^ int(code[-j])
        error_pos += val * (2 ** i)

    if error_pos:
        print(f"Error found at position: {error_pos}")
        corrected_code = code[:n - error_pos] + str(1 - int(code[n -
error_pos])) + code[n - error_pos + 1:]
        return corrected_code

    else:
        print("No error detected.")
        return code

def main():
    original_data = input("Enter the data bits: ")
    m = len(original_data)
    p = calculate_parity_bits(m)
    positioned_data = position_parity_bits(original_data, p)
    hamming_code = calculate_parity_values(positioned_data, p)
    print(f"Hamming Code: {hamming_code}")
```

```
    received_code = input("Enter the received Hamming Code: ")
    corrected_code = detect_correct_error(received_code, p)
    print(f"Corrected Hamming Code: {corrected_code}")


if __name__ == "
    main__": main()
```

Output

```
Enter the data bits:
1001010 Hamming Code:
10011010001
Enter the received Hamming Code:
10011010001 No error detected.
Corrected Hamming Code: 10011010001
```

Observation

1. Data expands from 4 to 7 bits with check bits
2. Check bits at positions 1, 2, and 4
3. Each check bit covers overlapping data groups
4. Even parity ensures balanced 1s
5. Final encoded output: **0110011**

Result

Thus, Hamming code successfully generated with even parity.